

Farm하니 v2 애플리케이션 소개서

항목	내용
문서명	Farm하니 v2 애플리케이션 소개서
프로젝트	SKN30 4차 단위 프로젝트 · Farm하니 v2
문서 버전	1.0
작성일	2026-08-03
기준 브랜치	<code>develop</code>
배포 URL	https://farmhani.vercel.app/
관련 문서	요구사항 정의서 · 화면설계서 · 시스템 구성도 · 통합테스트 계획서/결과보고서
문서 상태	4차 프로젝트 산출물

1. 서비스 개요

1.1 한 줄 소개

Farm하니 v2 는 사용자가 등록한 식물·텃밭 정보와 사진을, 공식 농업 문서 기반 **RAG(검색 증강 생성)** 및 **다중 LLM** 과 결합해 근거 있는 관리 상담을 제공하는 웹 애플리케이션이다.

1.2 추진 배경

기존 식물 관리 서비스와 커뮤니티 상담은 답변까지 시간이 걸리고, 답변의 근거나 정확성을 사용자가 확인하기 어렵다. 일반적인 LLM 상담 역시 사용자의 실제 식물 정보·관리 이력·공식 문서 근거가 연결되지 않으면 환각이나 부정확한 처방을 생성할 위험이 있다.

Farm하니 v2는 다음 문제를 해결한다.

- 사용자의 실제 관리 맥락이 반영되지 않는 일반적 답변
- 출처가 없거나 질문과 무관한 문서를 사용한 답변
- 식물 한 개만 상담할 수 있어 텃밭 전체 환경을 함께 판단하기 어려운 문제
- 단일 외부 LLM 장애·사용량 소진 시 상담을 계속할 수 없는 문제

1.3 핵심 가치

가치	설명
근거 우선	답변은 검색된 공식 문서와 출처(Citation)를 기준으로 생성하고, 근거가 없으면 "모른다"를 명시
내 데이터 결합	등록 식물·텃밭·재배 기록·사진을 상담 맥락에 반영
텃밭 통합 상담	식물 한 개 또는 텃밭 전체를 하나의 상담 맥락으로 전환
장애에 강한 구조	OpenAI 다중 모델 + 로컬 모델 Fallback으로 상담 연속성 확보
안전한 처방	병명 확정·직접 처방 대신 관찰 신호 중심 안내, 농약은 안전 고지 강제

1.4 대상 사용자

반려식물 또는 소규모 텃밭을 관리하는 일반 사용자, 식물명을 잘 모르는 초보 사용자, 여러 식물·텃밭을 함께 관리하는 다중 작물 사용자를 대상으로 한다.

2. 주요 기능

기능	설명
식물 프로필 관리	식물명·품종·위치 등록, 사진 업로드·갱신, 재배·물주기 기록, 선택 삭제
텃밭 통합 관리	단일/복합 작물 텃밭 CRUD, 등록 식물 배정, 텃밭 대표 사진
단일·텃밭 상담 전환	식물 한 개 또는 텃밭 전체를 하나의 상담 맥락으로 자유롭게 전환
전문가 상담 모드	객관적·행동 중심 말투로 상태 분석 및 관리 가이드 제공
내 식물과 대화 모드	식물이 1인칭으로 말하는 발랄하고 친근한 대화형 모드
멀티 LLM 선택	GPT-5.4 / 5.5 / 5.6 및 로컬 Qwen3-VL 중 상담 모델 선택
멀티모달 이미지 분석	사진에서 황화·반점·처짐 등 관찰 신호 추출 후 검색·답변에 반영
RAG 근거 제공	답변마다 참조한 공식 문서 출처(Citation) 표시
물주기 리마인더	종별 권장 간격 기반 오늘의 관리 체크리스트 제공
모바일 PWA	하단 내비게이션 · 채팅 우선 · 접이식 상담 설정 패널 · 설치형 지원

3. 시스템 개요 및 아키텍처

Farm하니 v2 전체 배포 구성

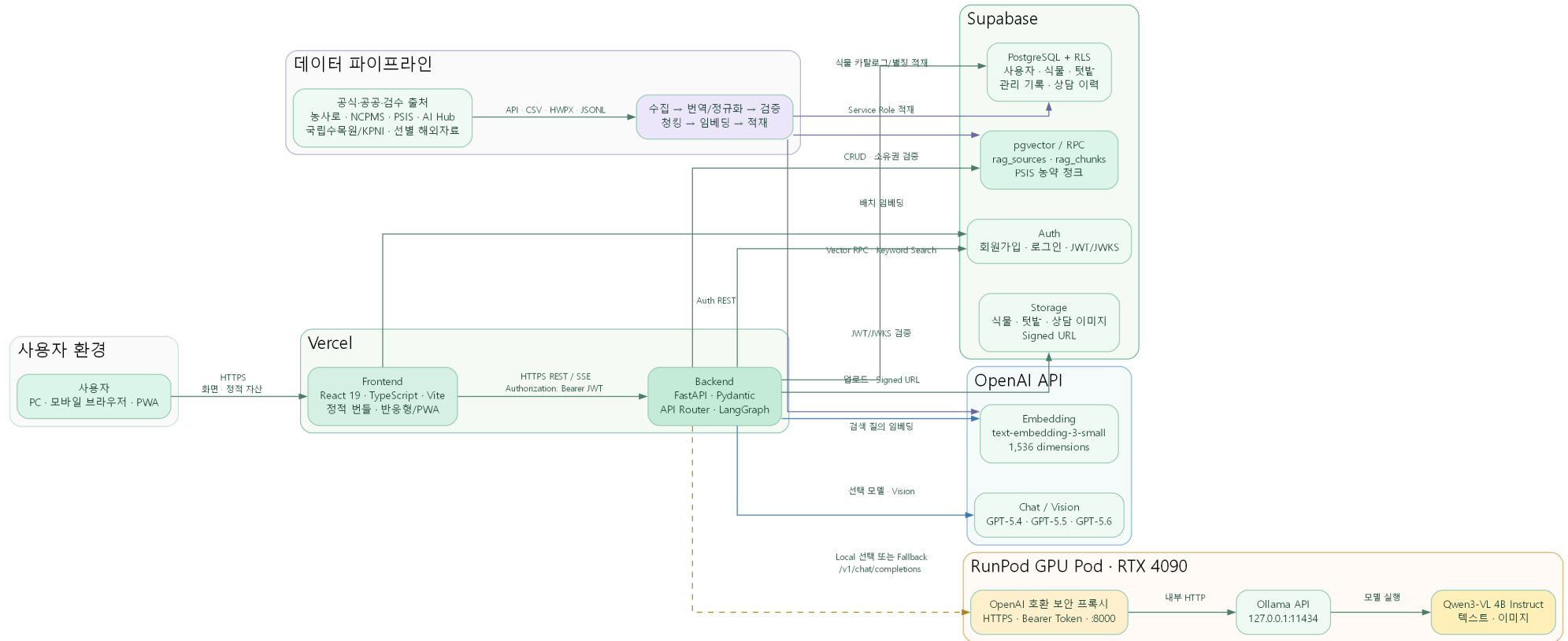


그림 1. Farm하니 v2 전체 배포 구성

3.1 구성 원칙

- **역할 분리** — UI, API, 인증·DB·Storage, LLM, 데이터 파이프라인을 독립 구성요소로 분리한다.
- **프론트/백엔드 분리 배포** — React 프론트엔드와 FastAPI 백엔드를 각각 별도 Vercel 프로젝트로 배포한다.

- **제공자 독립성** — 공통 OpenAI 호환 인터페이스와 모델 라우터로 OpenAI·로컬 모델을 전환한다.
- **안전한 실패** — 근거 0건, 모델 장애, 이미지 분석 실패를 정상 분기 또는 명확한 오류로 처리한다.

3.2 구성요소

영역	배포/기술	주요 책임
Frontend	Vercel · React 19 · TypeScript · Vite	화면·상태·반응형 UI, Supabase Auth 연결, 백엔드 호출, PWA
Backend	Vercel · Python · FastAPI · Pydantic	인증 검증, CRUD API, 업로드, LangGraph RAG, LLM 라우팅
인증·DB·Storage	Supabase	Auth, PostgreSQL, RLS, Storage, pgvector 벡터 검색
주 LLM	OpenAI API	GPT-5.4 / 5.5 / 5.6 — 채팅·Vision·임베딩
로컬 LLM	RunPod RTX 4090 · Ollama	Qwen3-VL 4B — 명시적 Local 선택 또는 OpenAI 장애 Fallback
데이터 파이프라인	개발·데이터 작업 환경	수집·번역·정규화·검증·청킹·임베딩·적재 스크립트

3.3 통신 및 보안 경계

- 전 구간 **HTTPS**, 사용자 요청은 **Bearer JWT**, 운영 CORS는 허용 프론트 도메인으로 제한한다.
- 백엔드는 Supabase JWKS 공개키로 JWT를 로컬 검증하고, 무효 토큰은 401 / 인증 서비스 장애는 503으로 구분한다.
- 비밀값(Service Role Key, OpenAI Key, 로컬 프록시 토큰)은 서버 환경변수로만 관리하고, 프론트에는 공개 키(Anon Key)만 노출한다.

4. 주요 기술 스택

영역	기술
Frontend	React 19 · TypeScript · Vite (해시 라우팅, 지연 로딩, PWA)
Backend	FastAPI · Pydantic (Python 3.11+), REST + SSE 스트리밍
AI / LLM (주)	OpenAI GPT-5.4 · GPT-5.5 · GPT-5.6 (채팅 · Vision)
AI / LLM (로컬)	Qwen3-VL 4B Instruct — RunPod GPU + Ollama, OpenAI 호환 프록시
임베딩	OpenAI <code>text-embedding-3-small</code> (1,536차원)
RAG 워크플로우	LangGraph StateGraph — 10단계 에이전틱 파이프라인
DB / Vector	Supabase PostgreSQL + pgvector (HNSW · cosine)
Auth / Storage	Supabase Auth (JWT/JWKS · RLS) · Supabase Storage (Signed URL)
배포	Vercel (Frontend) · Vercel (Backend) · Supabase · RunPod GPU

5. RAG 파이프라인 — LangGraph 10단계

Farm하니 v2 상담/RAG 런타임 흐름

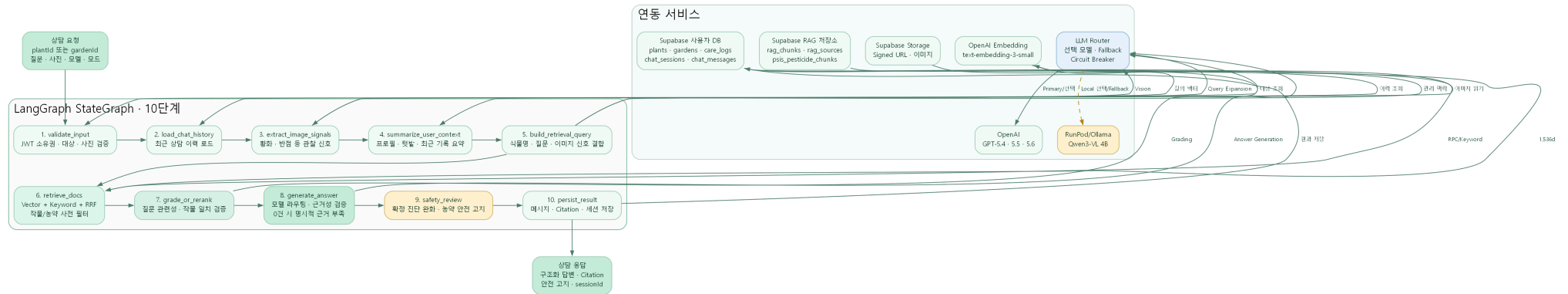


그림 2. LangGraph 10단계 상담/RAG 런타임 흐름

단순 1회성 벡터 검색이 아니라, 입력 검증부터 멀티모달 신호 병합, 동적 쿼리 생성, 문서 필터링, 안전성 검토까지 이어지는 에이전틱 워크플로우이다. 처리 단계는 SSE로 사용자에게 실시간 표시된다.

단계	노드	처리
1	validate_input	소유권·대상·사진 접근 검증
2	load_chat_history	최근 상담 메시지로 대화 맥락 구성
3	extract_image_signals	항화·반점·처짐 등 관찰 신호 추출
4	summarize_user_context	프로필·텃밭·관리 기록 요약
5	build_retrieval_query	식물명·증상 중심 검색 질의 확장
6	retrieve_docs	Vector + Keyword 검색, RRF 병합, 작물·농약 사전 필터
7	grade_or_rerank	관련성·식물 일치 여부 LLM 재평가
8	generate_answer	답변 생성, 근거성 검증, 근거 0건 처리
9	safety_review	확정 진단 완화, 농약 안전 고지 강제
10	persist_result	답변·Citation·세션 저장

5.1 검색 구조와 근거성

- **하이브리드 검색** — pgvector 유사도 검색과 키워드 검색을 병행하고 RRF(Reciprocal Rank Fusion)로 순위를 병합한다.
- **작물 게이트 · 농약 가드** — 질문 식물과 다른 작물 문서, 농약 의도가 없는 질문의 농약 문서를 사전 차단한다.
- **근거성(Grounding)** — 답변이 근거를 충분히 반영하지 못하면 문서 발췌 기반 답변으로 교체하고, 근거가 0건이면 생성을 건너뛰고 근거 부족을 명시한다.
- **평가** — LLM-as-a-Judge(12개 골든셋) 자동 평가에서 검색 정확도 3.58 → 4.83점, 사진 반영 만점(5.00)을 달성했다.

6. 멀티 LLM 및 멀티모달

6.1 선택 모델과 장애 대응

UI 표시	API 모델 ID	제공자
GPT-5.4	<code>gpt-5.4</code>	OpenAI
GPT-5.5	<code>gpt-5.5</code>	OpenAI
GPT-5.6	<code>gpt-5.6-sol</code>	OpenAI
Local	<code>qwen3-v1:4b-instruct</code>	RunPod / Ollama

- **선택형 Fallback** — OpenAI 호출이 timeout·5xx·키 미설정으로 실패하면 로컬 모델로 전환한다. 잘못된 요청 등 비재시도 오류는 Fallback 대상에서 제외한다.
- **Circuit Breaker** — 연속 실패가 임계값에 도달하면 일정 시간 OpenAI 호출을 건너뛰고 로컬로 직행한다. 임계값·유지 시간·응답 제한은 환경변수로 조정한다.
- **상태 정직성** — 모델 상태는 런타임에 실제 확인해 초록/빨강으로 표시하고, 두 제공자 모두 실패하면 사용 불가를 명확히 안내한다.
- **공통 규칙** — 선택 모델과 무관하게 동일한 RAG 근거·출처·안전 규칙이 적용된다.

6.2 멀티모달 이미지 처리

- OpenAI 경로는 Supabase Storage **Signed URL** 을 Vision 요청에 전달한다.
- 외부 URL 접근이 제한된 로컬 경로는 허용된 Supabase 호스트 이미지를 안전하게 내려받아 MIME·8MB 재검증 후 **Base64** 로 변환해 처리한다.
- 사진은 병명을 확정하지 않고 관찰 신호 중심으로 분석하며, 이미지 분석이 실패해도 텍스트·기록 기반 상담을 계속한다.

7. 데이터 파이프라인

Farm하니 v2 RAG 데이터 수집·가공·적재 흐름

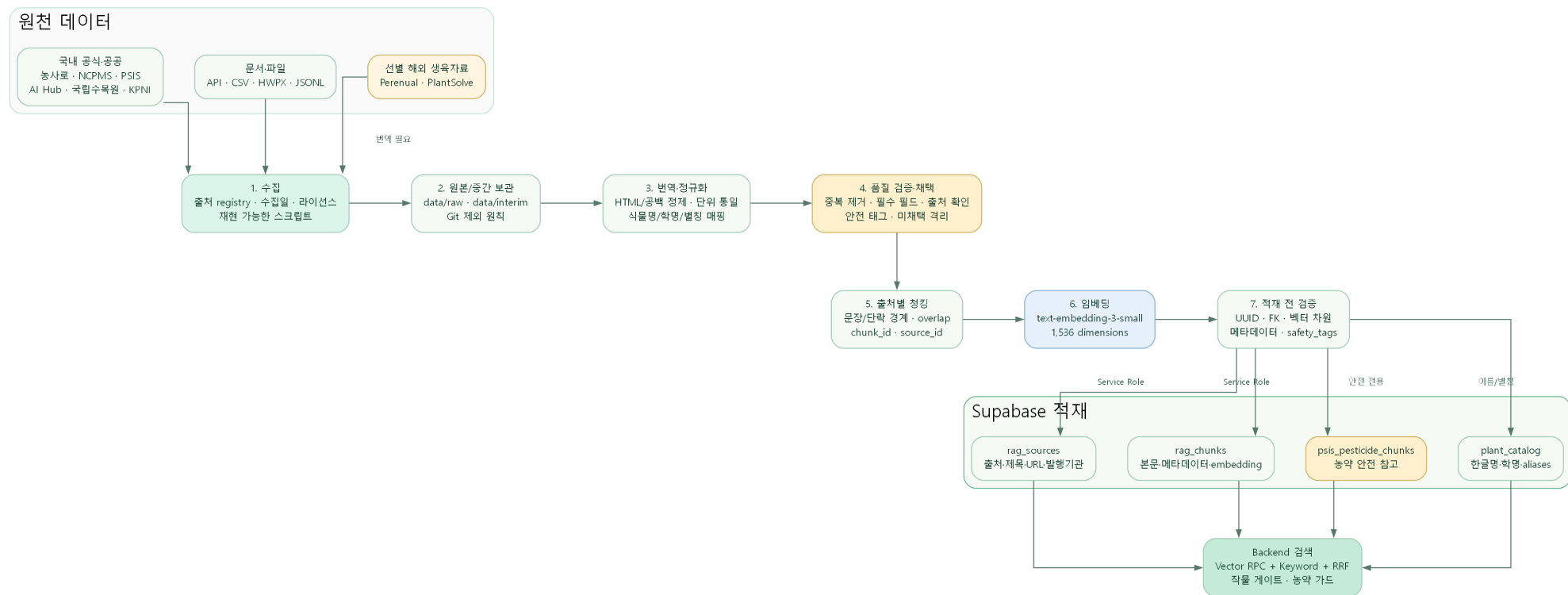


그림 3. 공식·공공·선별 해외자료의 수집·가공·적재 흐름

공공·공식 데이터 소스에서 수집·번역·정규화·검증을 거쳐 Supabase pgvector에 **총 4,841건의 청크**를 통합 적재했다.

출처	적재량	출처	적재량
NCPMS 병해충	2,152건	농업날씨365	100건
PSIS 농약 정보	1,268건	네이버 지식백과	49건
농사로	829건	PlantSolve (해외)	28건
국립수목원 도감	272건	합계	4,841건
AI Hub	143건		

- **가변 청킹** — 출처별 문서 성격에 맞춰 청크 크기를 조정한다. NCPMS 1,400자 / overlap 160자, PSIS 2,200자 / 250자, 농사로 1,200자 / 140자, 국립수목원 약 1,400자 / 160자.
- **선별 채택** — 해외 자료 PlantSolve는 원본 98건 중 품질 검증을 통과한 28건만 채택했고, 고유성이 낮고 농약 일반화 위험이 있는 EPPO·Perennial은 전량 제외했다.
- **안전 태그** — 병해충·농약 문서에는 `pesticide_caution`, `expert_check_required` 등 안전 태그와 사용 범위(`usage_scope`)를 주입한다.

8. 데이터베이스 구조

Supabase PostgreSQL 위에 사용자 데이터, 상담 데이터, RAG 문서, 식물 도감을 구성한다. 벡터 검색은 pgvector와 HNSW 인덱스(cosine)를 사용한다.

8.1 테이블 카탈로그

테이블	주요 컬럼	역할
profiles	id(=auth.users), username, full_name, avatar_url	사용자 프로필 (Auth와 1:1, 가입 시 트리거 자동 생성)
gardens	id, user_id, name, cultivation_type(single/mixed), representative_crop, image_url	텃밭 · 재배 유형 · 대표 작물
plants	id, user_id, garden_id, name, species, health_score, next_task, image_url	사용자 식물 프로필 · 텃밭 소속
care_logs	id, plant_id, watered_at, leaf_condition, soil_condition, memo	물주기·관찰·관리 기록
plant_photos	id, plant_id, garden_id, storage_path, captured_at	식물/텃밭 사진 메타데이터
chat_sessions	id, user_id, plant_id, garden_id, response_mode(expert/companion), title	단일 식물·텃밭 상담 세션
chat_messages	id, session_id, role(user/assistant), content(JSONB), citations(JSONB)	상담 대화 이력 · 출처
chat_feedback	id, message_id, user_id, rating, comment	AI 답변 피드백 (helpful/not_helpful/unsafe/irrelevant)
plant_catalog	id, name, species, aliases(text[]), watering_interval_days	식물 품종 도감 (한글명·학명·이명·권장 물주기)
rag_sources	source_id, title, url, publisher, collected_at	RAG 문서 출처
rag_chunks	chunk_id, source_id, text, embedding(vector 1536), crop_or_plant, metadata	공식 문서 청크 · 임베딩
psis_pesticide_chunks	chunk_id, source_id, text, embedding(vector 1536), metadata	농약 안전 참고 전용 청크

8.2 주요 관계

```
auth.users —1:1— profiles
profiles(user_id) —1:N— gardens, plants, chat_sessions
gardens(id) —1:N(SET NULL)— plants      (텃밭 삭제 시 식물은 소속만 해제)
plants(id) —1:N(CASCADE)— care_logs, plant_photos
gardens(id) —1:N(CASCADE)— plant_photos  (사진은 식물/텃밭 중 정확히 하나에 귀속)
chat_sessions(id) —1:N(CASCADE)— chat_messages
chat_messages(id) —1:N(CASCADE)— chat_feedback
rag_sources(source_id) —1:N— rag_chunks
```

8.3 무결성 및 제약

- `plant_photos` 는 `CHECK (num_nonnulls(plant_id, garden_id) = 1)` 로 식물 또는 텃밭 중 **정확히 하나**에만 귀속된다.
- `gardens.cultivation_type` 은 `single / mixed`, `chat_sessions.response_mode` 는 `expert / companion`, `chat_messages.role` 은 `user / assistant` 로 값이 제한된다.
- 텃밭 삭제 시 `plants.garden_id` 는 `SET NULL` 로 소속만 해제되고 식물 자체는 유지된다.
- `chat_feedback` 은 `UNIQUE(message_id, user_id)` 로 메시지당 사용자 1회 피드백을 보장한다.

8.4 보안 (RLS)

- 모든 사용자 데이터 테이블에 **Row Level Security** 를 활성화하고, `auth.uid()` 기준으로 본인 데이터만 CRUD를 허용한다.
- 하위 자원(`care_logs`, `plant_photos`, `chat_messages`, `chat_feedback`)은 상위 소유자(식물·텃밭·세션) 소유권을 따라 접근을 제한한다.
- `rag_sources`, `rag_chunks`, `plant_catalog` 은 공개 읽기 전용(Select Only)이며 쓰기는 Service Role로 제한한다.

8.5 RAG 벡터 검색

- 검색은 `match_rag_chunks(query_embedding, match_threshold, match_count)` RPC로 수행하며, 코사인 유사도(`1 - (embedding <=> query)`)로 후보를 반환한다.
- 이 RPC는 `rag_chunks` 와 `psis_pesticide_chunks` 를 각 테이블 상위 N개로 좁힌 뒤 **UNION** 하여 함께 검색한다. 농약 데이터는 갱신·등록취소가 잦아 사본을 복제하지 않고 단일 진실원천으로 유지한다.
- `rag_chunks.embedding`, `psis_pesticide_chunks.embedding` 에 HNSW(cosine) 인덱스, `rag_chunks.text` 에 trigram 인덱스, `crop_or_plant` 에 GIN 인덱스를 두어 검색 속도를 확보한다.

9. 화면 구성

React 컴포넌트·상태 기반으로 재설계했으며, PC 헤더와 모바일 하단 내비게이션을 포함한 통합 반응형 구조를 가진다.

화면 ID	화면명	핵심 기능
SCR-01	로그인/회원가입	로그인·회원가입·서비스 소개
SCR-02	대시보드/내 식물	식물 현황, 체크리스트, 검색, 선택 삭제
SCR-03	식물 등록	기본정보, 재배환경, 텃밭 배치, 첫 사진
SCR-04	식물 상세	프로필, 사진, 관리 기록, 정보 수정, 상담 이동
SCR-05	내 텃밭	텃밭 CRUD, 사진, 재배 유형, 식물 배치, 텃밭 상담
SCR-06	AI 상담 (PC)	단일·텃밭 상담, 모델 선택, 사진·텍스트 질문, 출처
SCR-07~09	모바일 대시보드·텃밭·상담	채팅 우선 배치, 하단 내비게이션, 접이식 설정 패널

10. 배포 및 운영

항목	내용
프론트엔드	Vercel 정적 배포 (<code>vercel.json</code>) — <code>farmhani.vercel.app</code>
백엔드	별도 Vercel FastAPI 프로젝트 (<code>backend/vercel.json</code> , <code>backend/index.py</code>)
상태 확인	<code>GET /health</code> , <code>GET /api/v1/chat/model-info</code> (모델 런타임 상태)
환경 설정	URL·모델·키·timeout·Fallback을 코드가 아닌 환경변수로 관리
품질 검증	백엔드 자동 테스트 187건 통과, 프론트 TypeScript·Vite 프로덕션 빌드 통과

11. 요약

Farm하니 v2는 공식 농업 문서 RAG를 중심으로, 사용자의 실제 식물·텃밭 데이터와 멀티모달 이미지, 다중 LLM을 결합한 근거 기반 식물 상담 서비스이다.

React·FastAPI·Supabase·OpenAI·RunPod로 구성된 분리형 클라우드 아키텍처 위에서, LangGraph 10단계 파이프라인이 검색·근거성·안전성을 단계적으로 보장한다. 4,841건의 공식 문서 청크와 RLS 기반 사용자 격리, 로컬 모델 Fallback으로 신뢰성과 안전성, 가용성을 함께 확보한 것이 핵심이다.

12. 변경 이력

버전	날짜	변경 내용
1.0	2026-08-03	Farm하니 v2 애플리케이션 소개서 최초 작성